

操作系统实验文档

Lab 11 - CPU Scheduling

eb4d26b1

Built by github-actions, at 2025-04-28 16:45:12+08:00

1. Weekly Lab Material

1.1 CPU Scheduling

1.1.1 Experiment Objectives

1. Master the xv6 scheduling process
2. Master the specific implementation of the Round-Robin scheduling algorithm

xv6-lab9 Code Branch

<https://github.com/yuk1i/SUSTech-OS-2025/tree/xv6-lab9>

Use the command `git clone https://github.com/yuk1i/SUSTech-OS-2025 -b xv6-lab9 xv6lab9` to download the xv6-lab9 code.

Use `make run` to run the kernel for this Lab, which will start the first user process `init`, and `init` will start the Shell process `sh`. In `sh`, enter `schedtest` to run the user program and observe the results.

In this experiment scenario, we will enable the timer interrupt. The timer interrupt will trigger at set intervals, and the timer interrupt handler will schedule the next user process. Multiple user processes will run in a Round-Robin manner, with each process executing for one time slice until completion.

1.1.2 Timer Interrupt

We have already learned about timer interrupts in the Week 4 lab session.

One thing to note is that we need to trigger a timer interrupt at regular intervals, but the `sbi_set_timer()` interface can only set one timer interrupt at a time. Therefore, we set the first timer interrupt during initialization, and in each timer interrupt handler, we set the next timer interrupt.

The preparation for the timer interrupt includes the initialization of the timer interrupt `timer_init()` and the setting of the next timer interrupt `set_next_timer()`:

```
//os/timer.c

/// Enable timer interrupt
void timer_init() {
    // Enable supervisor timer interrupt
```

```

    w_sie(r_sie() | SIE_STIE);
    set_next_timer();
}

/// Set the next timer interrupt
void set_next_timer() {
    const uint64 timebase = CPU_FREQ / TICKS_PER_SEC;
    if (on_vf2_board) {
        set_timer(get_cycle() + timebase);
    } else {
        w_stimecmp(r_time() + timebase);
    }
}

```

After the operating system starts, it will call `timer_init()` to enable the timer interrupt and set the first timer interrupt:

```

//os/main.c

static void bootcpu_init(){
    //.....

    timer_init();

    //.....
}

```

During the execution of a user process, once a timer interrupt is triggered, it will jump to the trap handling process `usertrap()` to handle the timer interrupt.

When handling the timer interrupt, the next timer interrupt is set, and `which_dev` is set to 1, where 1 indicates that scheduling is needed. Afterward, at the end of the trap handling function, the value of `which_dev` is checked. If it is 1, `yield()` is called to relinquish CPU resources and schedule the next process.

```

//os/trap.c

void usertrap() {
    //.....

    if (cause & SCAUSE_INTERRUPT) {
        which_dev = handle_intr();
    }
    //.....

    // If it's a timer interrupt, call yield to give up CPU.
    if (which_dev == 1)
        yield();
    //.....
}

static int handle_intr(void) {
    uint64 cause = r_scause();
    uint64 code = cause & SCAUSE_EXCEPTION_CODE_MASK;
    if (code == SupervisorTimer) {

```

```

    tracef("Timer interrupt!");
    if (cpuid() == 0) {
        acquire(&tickslock);
        ticks++;
        wakeup(&ticks);
        release(&tickslock);
    }
    set_next_timer(); // Set next timer
    return 1;
} else if (code == SupervisorExternal) {
    tracef("External interrupt from usertrap!");
    plic_handle();
    return 2;
} else {
    return 0;
}
}

```

Scheduling Algorithm -- Round Robin

In `sched.c`, we have a queue `task_queue` that manages processes in the `RUNNABLE` state. This linked list is called the ready queue.

In xv6, a process will be set to `Runnable` and enqueued to the tail of the ready queue via `add_task()` at the following moments: - fork - exec -> load_init_app - wakeup - yield -> sched -> scheduler

When the scheduler retrieves the ready process `p` at the head of the queue via `fetch_task`, the process is removed from the ready queue, and its state is changed to `RUNNING`.

Since we always add processes to the tail of the ready queue and select the next ready process from the head, we have implemented the Round-Robin scheduling algorithm.

```

//os/sched.c
void scheduler() {
    //.....

    for (;;) {
        // Interrupts may be enabled here.

        p = fetch_task();
        //.....

        acquire(&p->lock);
        assert(p->state == RUNNABLE);
        debugf("Switch to process %d(%d)", p->index, p->pid);
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->sched_context, &p->context);

        // When we get back here, someone must have called swtch(..., &c->sched_context);
        assert(c->proc == p);
        assert(!intr_get()); // Scheduler should never have interrupts enabled
        assert(holding(&p->lock)); // Whoever switched to us must acquire p->lock
    }
}

```

```

        c->proc = NULL;

        if (p->state == RUNNABLE) {
            add_task(p);
        }
        release(&p->lock);
    }
}

void sched() {
    //.....
    swtch(&p->context, &mycpu()->sched_context);
    //.....
}

// Give up the CPU for one scheduling round.
void yield() {
    struct proc *p = curr_proc();
    debugf("Yield: (%d)%p", p->pid, p);

    // lab9 CPU scheduling:
    infof("Yield: %d", p->pid);

    acquire(&p->lock);
    p->state = RUNNABLE;
    sched();
    release(&p->lock);
}

```

? Question

Combining the content of this week and Week 5, summarize which functions of which processes are involved when a timer interrupt triggers a switch from one process to another.